UNIVERSITY OF LIMPOPO

Faculty of Science and Agriculture
Department of Computer Science

SCOA031 - Database Administration

Project 1: Enterprise Database Management & Monitoring System

Documentation Report

Due Date: 04 May 2026
Internal Examiner: Dr J Tlouyamma

SURNAME	NAME	STUDENT NUMBER
SEOLOANA	DIMAKATSO	202330079
MAGOAI	MANTHIBU	202108908
MONGWE	NWAYITELO HARHI	202306570
MUKHANINGA	MUVULEDZI	202220298
MOLEPO	TLHOLOGELO	202115821
BANDA	NKULULEKO MAGNIFICENT ALISAK	202232318
MADIBE	EVELYN MANARE	202415417
THOBANE	THABISO JIM	202316364
MODIBA	ROSINA MMATHOTO	202135584
KGANYAGO	MAPASEKA	202321810
MAKULA	TSHEPO LETLALO	202247878
MOTHA	NHLALONHLE	202136273

1. System Design - Explanation of All Components

This project required us to act as Database Administrators (DBAs) for a company running on the AdventureWorks2022 database. The goal was to build a complete database management system that covers automation, security, performance, auditing, backups, and maintenance. Below is an explanation of each part we implemented. The AdventureWorks2022 database was used as the foundation for all tasks in this project (Microsoft Learn, 2026a).

1.1 Part A - Stored Procedures (Database Automation)

We created six stored procedures in total. Three were for data management and three were for reporting.

Data Management Procedures

usp_AddNewCustomer - This procedure adds a new customer to the database. It accepts inputs like first name, last name, email, phone, and address. Before inserting anything, it checks that required fields are not empty and that the email is not already in the system. It then inserts records into several related tables (Person.BusinessEntity, Person.Person, Person.EmailAddress, etc.) all inside a single transaction so that if any step fails, everything gets rolled back using TRY...CATCH.

usp_UpdateProductPrice - This procedure updates the list price of a product. You can pass either a ProductID or a product name. It validates that the new price is not negative and that the product actually exists. It also checks that you are not setting the same price that already exists. A transaction wraps the update to make sure it is safe.

usp_ArchiveInactiveCustomers - This procedure finds customers who have not placed any orders within a given number of months (default is 36). It moves them into an archive table and then deletes them from the main Sales.Customer table. It also has a preview mode so you can see who would be affected before committing.

Reporting Procedures

usp_MonthlySalesReport - Generates a monthly breakdown of sales for a given date range. It shows total orders, unique customers, revenue, and average order value grouped by month.

usp_Top10BestSellingProducts - Returns the top 10 products by units sold within a date range. Includes revenue, average selling price, and how many times each product was ordered.

usp_EmployeePerformanceSummary - Summarises each salesperson's performance within a date range. Shows total sales, quota attainment percentage, and a rank compared to other salespeople.

1.2 Part B - User Roles & Security

We created three database roles and mapped users to them. The roles control what each user can and cannot do inside the database. Role-based access control is a standard approach in SQL Server for managing user permissions at the schema and object level (Microsoft Learn, 2025a). More detail is in Section 2 (Security Model).

1.3 Part C - Performance Optimization

We identified slow queries and created indexes to speed them up. We also created three monitoring stored procedures that help the DBA keep an eye on performance. SQL Server provides built-in dynamic management views (DMVs) and tools for monitoring and tuning query performance (Microsoft Learn, 2025b). More detail is in Section 3 (Performance Discussion).

1.4 Part D - Backup & Recovery

We wrote SQL scripts to create full, differential, and transaction log backups of the AdventureWorks2022 database. SQL Server supports multiple backup types to enable flexible recovery strategies depending on the business requirements (Microsoft Learn, 2026b). We also demonstrated a restore process using a test database called AdventureWorks_RestoreTest. More detail is in Section 4 (Backup & Recovery).

1.5 Part E - Triggers, Logging & Email Alerts

We created an audit table called dbo.AuditTable and wrote triggers on Sales.Customer and Production.Product to automatically log all INSERT, UPDATE, and DELETE operations. SQL Server triggers make use of the virtual *inserted* and *deleted* tables to access the rows affected by each data modification operation (Microsoft Learn, 2025c). We also configured SQL Server Database Mail to send email alerts to the DBA when important events happen, such as a record being deleted or a product price changing by more than 10% (Microsoft Learn, 2026c).

1.6 Part F - Maintenance & Server Health

We created a maintenance stored procedure called usp_RunDatabaseMaintenance that runs DBCC CHECKDB for integrity checks and handles index rebuilding and reorganising based on fragmentation levels. We then used SQL Server Agent to schedule this procedure to run automatically every Sunday at 1:00 AM.

2. Security Model - Roles and Permissions

2.1 Roles Created

We created three custom database roles inside AdventureWorks2022. In SQL Server, database roles allow the DBA to group users together and assign permissions to the role rather than to individual users, which simplifies permission management (Microsoft Learn, 2025a):

Role Name	Purpose
SalesRole	For sales staff who need to read and write sales data
HRRole	For HR staff who only need to view employee information
DBA_Role	For database administrators who need full control

2.2 Permissions Assigned

Each role was given specific permissions using GRANT statements. The principle of least privilege was applied, meaning each role was only given the permissions it actually needs (Microsoft Learn, 2025a):

SalesRole

- GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Sales - full read/write on the Sales schema
- GRANT SELECT ON SCHEMA::Person - read-only access to customer/person data
- GRANT SELECT ON SCHEMA::Production - read-only access to product data

HRRole

- GRANT SELECT ON SCHEMA::HumanResources - read-only access to employee records
- GRANT SELECT ON Person.Person and Person.EmailAddress - access to names and emails only

DBA_Role

- GRANT CONTROL - full administrative control over the database

2.3 Users Created and Mapped

We created server logins and database users and then assigned each user to the appropriate role:

Login Name	Database User	Role Assigned
Emmanuel(Sales_2)	Emmanuel_(Sales_2)	SalesRole
MolepoT_(HR_1)	HR_2	HRRole
Butcher(DBA_3)	Administrator_3	DBA_Role

2.4 Security Testing

To test that the roles worked, we used EXECUTE AS USER to simulate each user's session and then ran queries. For example:

- When running as the SalesRole user, we could SELECT from Sales.SalesPerson successfully.
- When running as the HRRole user, we could access HumanResources.Employee but not Sales tables.
- When running as the DBA_Role user, we could query system objects like sys.objects.

The REVERT command was used after each test to switch back to the original session.

3. Performance Improvements - Before vs After

3.1 Why We Needed Indexes

The AdventureWorks database has several large tables. Without proper indexes, queries that filter by date, product name, or sales person have to scan the entire table row by row. This is called a Table Scan or Index Scan and it is very slow when dealing with thousands of records. SQL Server provides tools such as SET STATISTICS IO and execution plans to measure and compare query performance before and after applying optimisations (Microsoft Learn, 2025b).

3.2 Indexes Created

Index Name	Table	Purpose
IX_SalesOrderHeader_OrderDate_TotalDue	Sales.SalesOrderHeader	Speeds up Query 1 - filtering orders by date range
IX_Product_Name_ListPrice	Production.Product	Speeds up Query 2 - searching products by name prefix like 'Mountain%'
IX_SalesOrderHeader_SalesPersonID_OrderDate	Sales.SalesOrderHeader	Speeds up Query 3 - grouping sales by salesperson
IX_SalesOrderHeader_CustomerID	Sales.SalesOrderHeader	Speeds up Query 4 - customer purchase aggregation

We also dropped two redundant indexes (IX_Customer_CustomerID_PersonID and IX_Person_BusinessEntityID_Name) because they duplicated the clustered key or offered no benefit and only added overhead on writes (Microsoft Learn, 2025b).

3.3 Query Optimization

We rewrote four queries to be more efficient:

- Query 1 (Sales by date range): Changed the date comparison to use open-ended ranges (>= and <) instead of BETWEEN so the index could be used with a seek instead of a scan.
- Query 2 (Product search): Added an index that covers the Name and ListPrice columns so the engine does not need to look up the base table.
- Query 3 (Employee performance): Used a subquery to pre-aggregate order data before joining, which reduces the number of rows the engine has to process.
- Query 4 (Customer history): Used a CTE to pre-filter and aggregate orders first (using HAVING COUNT > 2) before joining to Person.Person, reducing the join size significantly.

3.4 Monitoring Stored Procedures

We created three procedures to help the DBA monitor performance. These procedures make use of SQL Server's dynamic management views, which provide real-time information about server state and performance (Microsoft Learn, 2025b):

- usp_DetectLongRunningQueries - queries sys.dm_exec_requests to find sessions running longer than a threshold (default 30 seconds). Shows the actual query text and execution plan.
- usp_CheckDatabaseSize - uses sp_spaceused and sys.database_files to show total database size, used space, and file paths.
- usp_MonitorIndexFragmentation - uses sys.dm_db_index_physical_stats to find indexes with fragmentation above a threshold and recommends either REORGANIZE (10-30%) or REBUILD (>30%).

We also created a usp_PerformanceDashboard procedure that calls all three monitoring procedures in one go, making it easy for the DBA to get a complete picture quickly.

4. Backup Strategy - Schedule and Recovery Plan

4.1 Backup Types Implemented

Before running any backups, we set the database recovery model to FULL using ALTER DATABASE AdventureWorks2022 SET RECOVERY FULL. This is required for transaction log backups to work properly. SQL Server supports three main backup types that together provide a comprehensive recovery strategy (Microsoft Learn, 2026b):

Backup Type	File	Purpose
Full Backup	AW2022_Full.bak	Complete snapshot of the entire database
Differential Backup	AW2022_Diff.bak	Only captures changes since the last full backup
Transaction Log Backup	AW2022_Log.trn	Captures all committed transactions since last log backup

4.2 Recommended Backup Schedule

Backup Type	Frequency	Time
Full Backup	Weekly (Sunday)	1:00 AM
Differential Backup	Daily (Mon-Sat)	11:00 PM
Transaction Log Backup	Every 4 hours	Throughout the day

4.3 Restore Process

To demonstrate recovery, we restored the backups to a new database called AdventureWorks_RestoreTest. The restore sequence follows the standard SQL Server recovery chain (Microsoft Learn, 2026b). The process follows these steps:

- Step 1: Restore the full backup with NORECOVERY (leaves the database in a restoring state so more backups can be applied).
- Step 2: Restore the differential backup with NORECOVERY.
- Step 3: Restore the transaction log backup with RECOVERY (this brings the database online and makes it usable).

The MOVE option was used to place the restored database files in a different location so they do not conflict with the original database files. The REPLACE option was used to overwrite an existing test database if it already existed.

5. Trigger & Alert Design - How Logging and Email Alerts Work

5.1 The Audit Table

We created a table called dbo.AuditTable with the following columns:

Column	Data Type	Purpose
AuditID	INT IDENTITY	Auto-generated primary key for each log entry
TableName	VARCHAR(128)	Name of the table that was changed
ActionType	VARCHAR(10)	Either INSERT, UPDATE, or DELETE
RecordID	VARCHAR(20)	The ID of the record that was changed
ActionUser	VARCHAR(128)	The SQL login that made the change
ActionDateTime	DATETIME	Timestamp of when the change happened
Description	VARCHAR(MAX)	A plain-English description of what happened

5.2 Triggers Created

We created one trigger per table, and each trigger handles all three operations (INSERT, UPDATE, DELETE). SQL Server triggers use the special *inserted* and *deleted* virtual tables to access the rows that were added or removed by the triggering statement (Microsoft Learn, 2025c). The action type is determined using IF EXISTS checks on these tables:

- Sales.trg_Customer_Audit - fires on Sales.Customer for INSERT, UPDATE, and DELETE
- Production.trg_Product_Audit - fires on Production.Product for INSERT, UPDATE, and DELETE

Inside each trigger, we used SET NOCOUNT ON to prevent row-count messages from interfering. We then determined the action type and inserted a record into AuditTable with a descriptive message. For example, a DELETE on Sales.Customer would produce a description like: 'The record that was deleted by sa belong to customer with CustomerId: 1' (Microsoft Learn, 2025c).

5.3 Email Alerts

We configured SQL Server Database Mail with a Gmail account and a profile called DBA_Alert_Profile. Database Mail uses the sp_send_dbmail stored procedure to send emails from within T-SQL code, including from inside triggers and stored procedures (Microsoft Learn, 2026c). The SMTP settings used were smtp.gmail.com on port 587 with SSL enabled. The sp_send_dbmail procedure is called inside the triggers to send emails when important events occur.

Email alerts are sent in the following situations:

- A record is deleted from Sales.Customer - sends a CRITICAL alert to the DBA
- PersonID is modified on Sales.Customer - treated as a sensitive data change and triggers a SECURITY ALERT
- A record is deleted from Production.Product - sends a product deletion alert

- A product's ListPrice is updated - conditionally sends an alert (see below)

5.4 Conditional Alerts (Advanced Logic)

Not all updates trigger an email. We only send alerts when the change meets a specific threshold:

- Price change > 10%: Inside the Product trigger, we compare the new and old ListPrice. If the difference divided by the old price is greater than 0.10, the email is sent. Otherwise it is ignored.
- Sensitive data modification: In the Customer trigger, we check UPDATE(PersonID). If PersonID was one of the columns modified, a security alert is sent because this could indicate fraudulent reassignment of a customer record.

All email calls are wrapped in TRY...CATCH so that if the mail server is unavailable, the trigger still completes and the audit log is still written (Microsoft Learn, 2026c). This ensures the triggers do not break normal database operations.

6. Summary

This project gave us practical experience in all the key areas of database administration. We built automation through stored procedures, secured the database with role-based access control (Microsoft Learn, 2025a), improved performance with targeted indexing and query rewrites (Microsoft Learn, 2025b), implemented a full backup and recovery strategy (Microsoft Learn, 2026b), tracked every change through audit triggers and email alerts (Microsoft Learn, 2025c), and set up automated maintenance using SQL Server Agent jobs. All work was implemented on the AdventureWorks2022 database (Microsoft Learn, 2026a).

References

Microsoft Learn (2026a) *AdventureWorks sample databases - SQL Server*. Available at: https://learn.microsoft.com/en-us/sql/samples/adventureworks-install-configure?view=sql-server-ver17 (Accessed: 27 April 2026).

Microsoft Learn (2026b) *SQL Server Database Backups*. Available at: https://learn.microsoft.com/en-us/sql/relational-databases/backup-restore/back-up-and-restore-of-sql-server-databases (Accessed: 1 May 2026).

Microsoft Learn (2026c) *Database Mail - SQL Server*. Available at: https://learn.microsoft.com/en-us/sql/relational-databases/database-mail/database-mail?view=sql-server-ver17 (Accessed: 1 May 2026).

Microsoft Learn (2025a) *Database roles (Database Engine)*. Available at: https://learn.microsoft.com/en-us/sql/relational-databases/security/authentication-access/database-level-roles (Accessed: 27 April 2026).

Microsoft Learn (2025b) *Performance Monitoring and Tuning*. Available at: https://learn.microsoft.com/en-us/sql/relational-databases/performance/performance-monitoring-and-tuning-tools (Accessed: 2 May 2026).

Microsoft Learn (2025c) *Use the inserted and deleted tables - SQL Server*. Available at: https://learn.microsoft.com/en-us/sql/relational-databases/triggers/use-the-inserted-and-deleted-tables?view=sql-server-ver17 (Accessed: 1 May 2026).